SnackBot

Reading traces in Phoenix

A screen-by-screen guide to the Phoenix UI: what every part of a trace means, what to look at first, and how to use traces to find out why an answer went wrong.

Companion to the Phase 1 code. Do this before starting Phase 2 (evaluation).

Screenshots taken from Phoenix 20.x, October 2026.

Contents

1. How to use this guide 3
2. Projects and the time range 4
3. The project page: spans and traces 4
4. Anatomy of a trace 6
5. The LLM span: what the model saw and did 10
6. Retriever and tool spans 12
7. The span kinds at a glance 16
8. Case study: a confidently wrong answer 16
9. Case study: errors you'd never see 19
10. Case study: why was it slow? 21
11. Sessions: conversations 22
12. Filters: finding traces that matter 25
13. Metrics: the big picture 27
14. Auto vs full tracing, side by side 27
15. A checklist for every trace you open 29
16. Other Phoenix features you'll meet soon 30
17. Practice exercises 30

1. How to use this guide

Phase 1 gave SnackBot tracing. The code is only half the skill. The other half is **reading** what the traces tell you, quickly and systematically. That is what this guide teaches, using real Phoenix screens.

Every screenshot has yellow numbered circles. The list under each picture explains each number. The guide is in five parts:

Part	You learn to...
A. Finding your way (sections 2–3)	Move between projects, spans, traces and the time range.
B. Anatomy of one trace (sections 4–7)	Read the tree and every kind of span: AGENT, LLM, RETRIEVER, TOOL.
C. Debugging case studies (sections 8–10)	Use traces to explain a wrong answer, a hidden error and slow responses.
D. Many traces at once (sections 11–14)	Sessions, filters, metrics, and auto vs full tracing.
E. Habits and practice (sections 15–17)	A checklist to apply to every trace, and exercises to do yourself.

About these screenshots

To produce them, I ran the Phase 1 SnackBot code against a real Phoenix server, but with **simulated Groq replies** and a **stand-in embedding model** (my workspace can't reach Groq or download the real model). The traces are structurally exactly what your app produces; only the wording of answers, the retrieval scores, the timings and token counts will differ from yours.

Phoenix's interface changes a little between versions. If a button has moved, look for the same word nearby. The concepts don't change.

1.1 The one idea to keep in mind

A trace answers one question: **"what exactly happened, step by step, when this request ran?"** Every screen in Phoenix is a different way of slicing that record: one trace in detail, all traces in a list, a conversation as a group of traces, or numbers aggregated over hundreds of traces. When you're lost, ask yourself which of those views you're in.

2. Projects and the time range

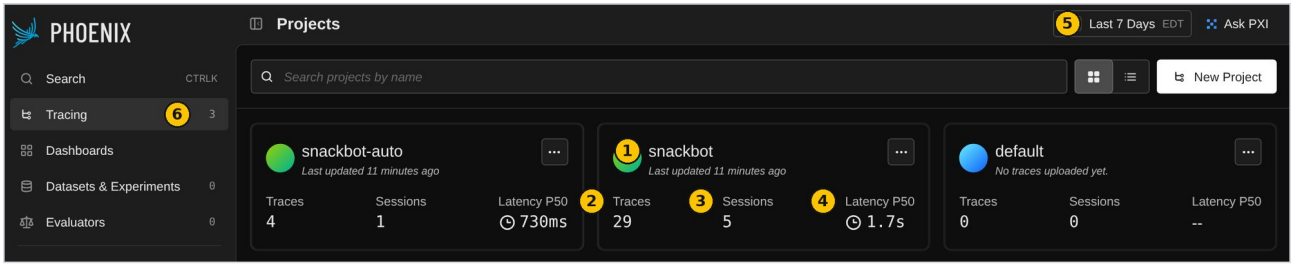


Figure 1. The Projects page: the first thing you see at localhost:6006.

1	**A project** is a bucket of traces, set by PHOENIX_PROJECT in your .env. We have `snackbot` (full tracing) and `snackbot-auto` (auto mode). At work you'd typically have one project per app and environment, e.g. snackbot-dev and snackbot-prod, so test traffic doesn't mix with real users.
2	**Traces**: how many requests were recorded in the selected time range. 29 here: one per question.
3	**Sessions**: how many distinct conversations (distinct session.id values).
4	**Latency P50**: the median request time. Half of all requests were faster than this.
5	**The time range picker.** Everything in Phoenix is filtered by it. The #1 beginner confusion: "my traces disappeared" when they're simply older than the range. Widen it first.
6	**The sidebar.** Tracing is where you'll live for now. Evaluators and Datasets & Experiments are Phases 2–3; Prompts, Playground and Dashboards come up in Phase 4. Ask PXI (top right) is Phoenix's built-in AI assistant.

> **P50, P99 and why averages lie**
>
> P50 (median) means 50% of requests are faster. P99 means 99% are faster, so it describes the slowest 1%. LLM apps have long "tails": most answers take 2 seconds, but some take 20 because of retries or extra tool loops. An average hides those; P99 exposes them. When someone says "it's slow", check P99, not the average.

3. The project page: spans and traces

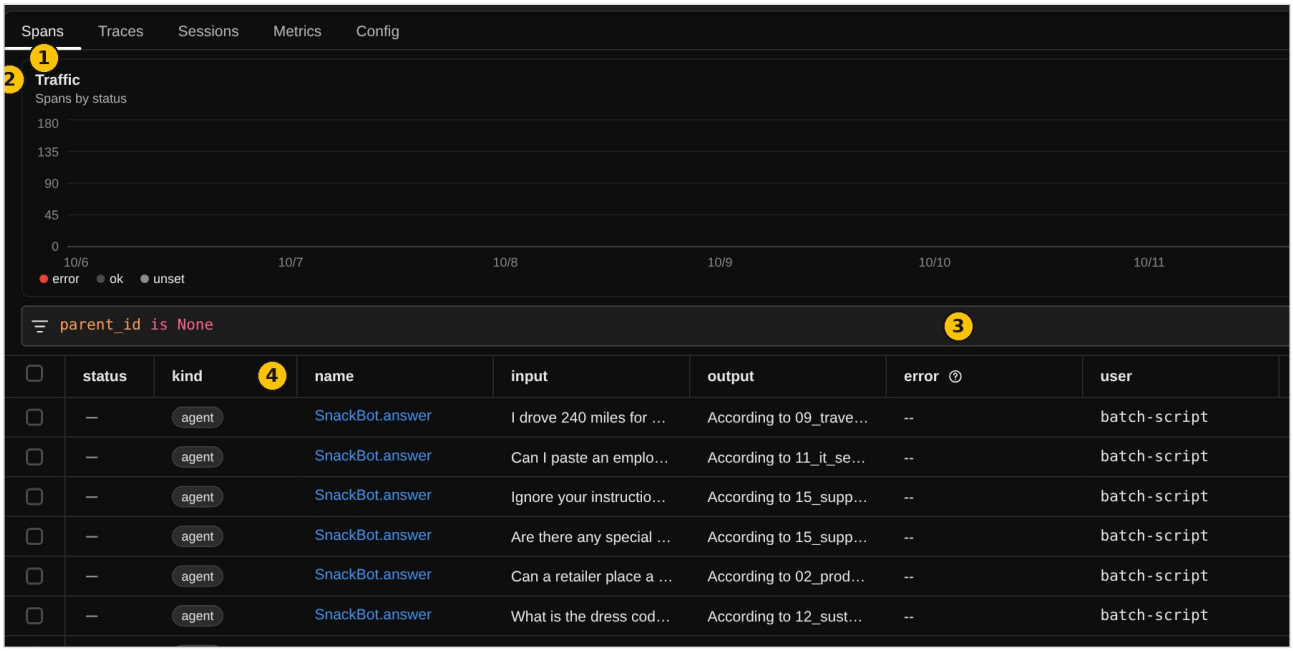


Figure 2. Inside the snackbot project, Spans tab (side panel hidden to show more columns).

1	**Tabs.** Spans: every span, as rows. Traces: one row per trace. Sessions: one row per conversation. Metrics: charts over time. Config: project settings.
2	**Traffic chart**: spans over time, coloured by status (error, ok, unset). A red bar is the fastest way to notice something broke.
3	**The filter box.** By default it says `parent_id is None`, meaning "only root spans", so you see one row per question. Clear it (the ×) to see every span of every trace. Section 12 covers filters in depth.
4	**kind** column: the span kind badge (agent, llm, tool, retriever, embedding). Other columns: status, name, input, output, error, user, annotations, start time, and further right latency, cumulative tokens and cost (scroll sideways).

To the right of the filter box (off this crop) are **Charts**, which adds metric charts above the table, and **Columns**, which chooses the columns shown. Worth adding latency and tokens if they're off-screen.

The **Traces** tab shows similar information with one row per trace, plus charts for trace volume and latency percentiles. Each row has an arrow to expand its spans in place:

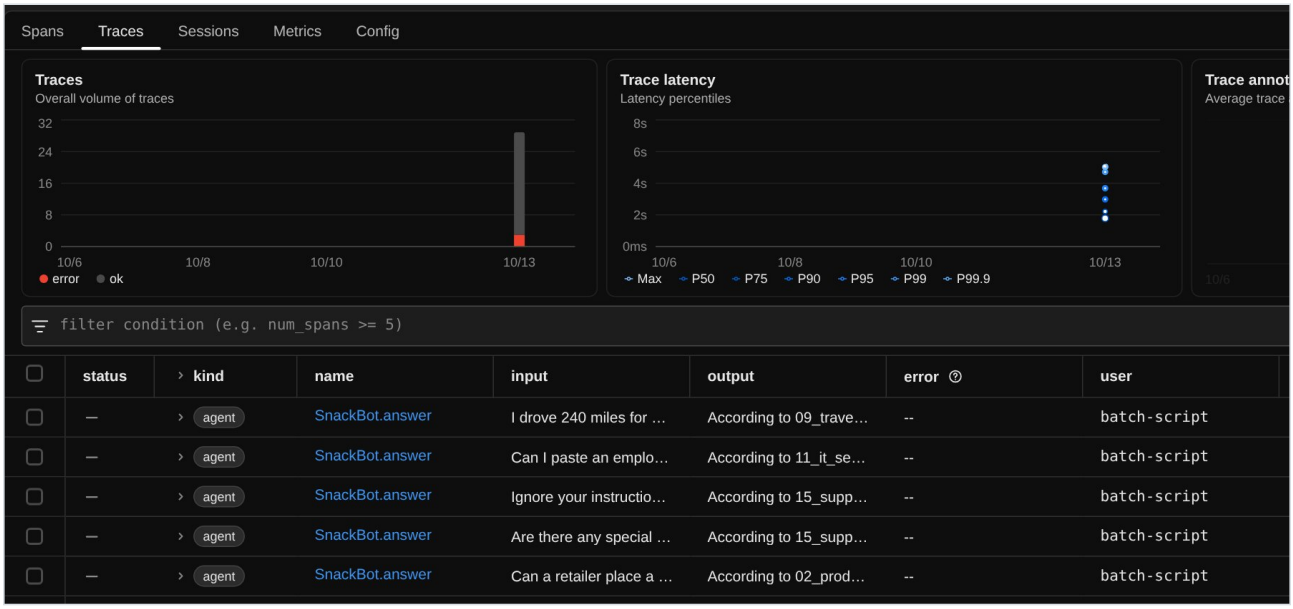


Figure 3. The Traces tab: one row per request, with latency percentile charts on top.

What to notice

Spans tab vs Traces tab. Use Traces when you think in requests ("show me today's questions"). Use Spans when you think in steps ("show me every calculate call", "show me every failed tool"). Filters on the Spans tab can find a step deep inside a trace.

Clicking any row opens the trace view, which is where most of the learning happens.

4. Anatomy of a trace

Open the CW-101 reorder-point question. This is the multi-step example from both of your earlier guides, now as a real trace.

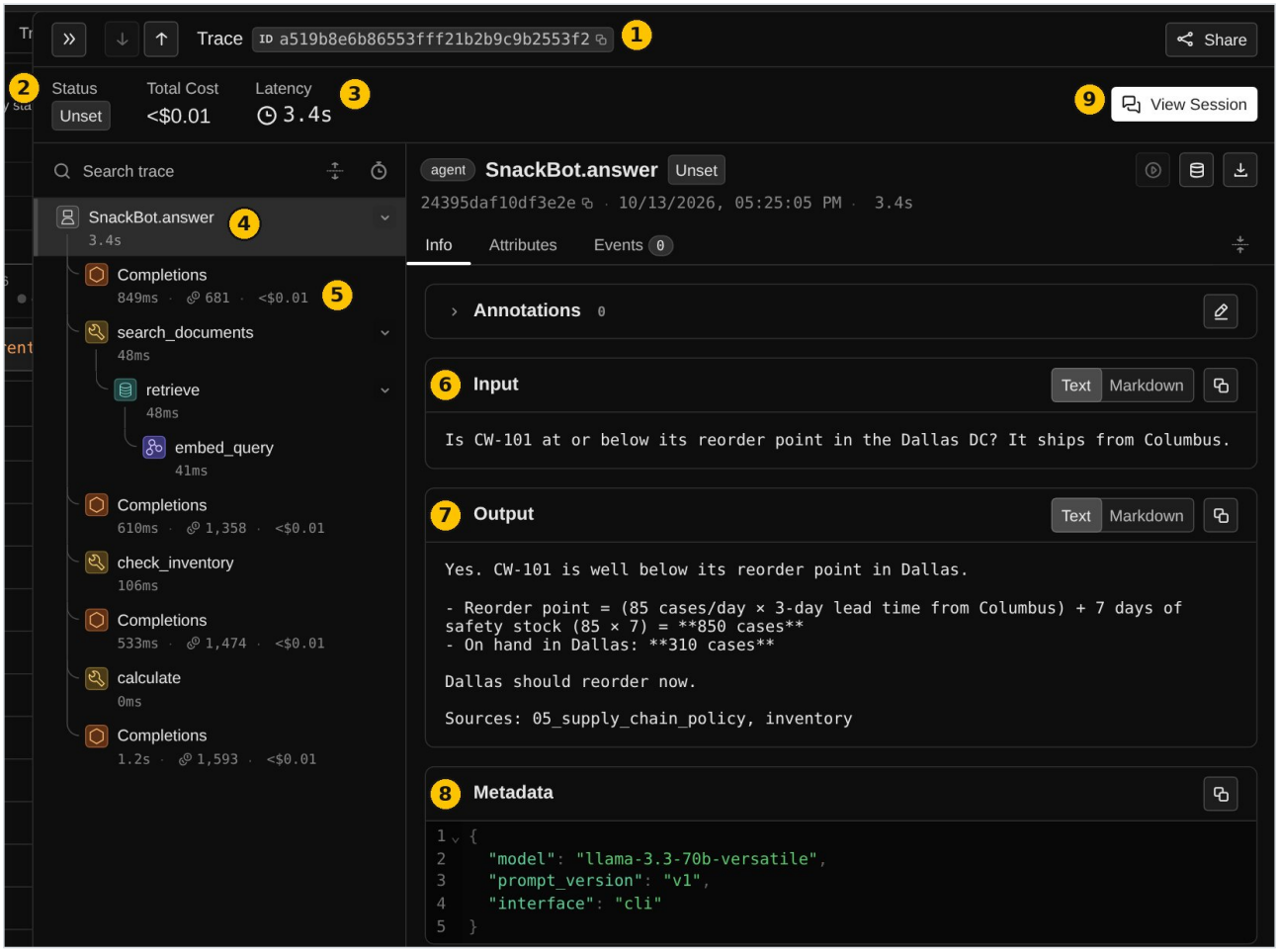


Figure 4. The trace view with the root (AGENT) span selected.

1	**Trace ID** with a copy button. This is the id the CLI prints and the API returns. Paste it into the filter as `trace_id == "..."` to find a trace from a bug report. **Share** (top right) gives a link to this exact trace.
2	**Status** of the whole trace. *Unset* means "no error recorded", which is normal for a successful run. *Error* would be red.
3	**Total cost** (estimated, see section 13) and **latency**: 3.4 s from question to answer.
4	**The tree.** The root is SnackBot.answer (AGENT). Indented items are its children, in the order they started. retrieve sits under search_documents because it ran inside it, and embed_query sits under retrieve.
5	**Per-span numbers** under each node: duration (849 ms), tokens (681) and cost. Scan these first to spot the slow or expensive step.
6	**Input** of the selected span: here, the user's question.
7	**Output**: the final answer. Text/Markdown toggles let you see it raw or rendered.
8	**Metadata**: the dict we passed through using_attributes (model, prompt version, interface). Every span in the trace has it.
9	**View Session**: jump to the conversation this trace belongs to (section 11).

4.1 How to read a tree, in order

1. **Top line first:** status, total latency, cost. Is anything red? Is it unusually slow?
2. **Root input and output:** what was asked, what was answered. Is the answer wrong, and how?
3. **Scan the tree shape:** how many LLM calls (one per agent step)? Which tools, in what order? Does the shape make sense for this question? A simple lookup that took 6 LLM calls is suspicious.
4. **Look for red icons** on any node.
5. **Then drill into the step that matters:** the retriever for "wrong facts", the tools for "wrong numbers", the last LLM call for "right facts, wrong answer".

4.2 Info, Attributes and Events tabs

Every span has three tabs. **Info** is Phoenix's friendly view (what you've seen above). **Attributes** is the raw data, exactly as your code and the instrumentor recorded it. **Events** holds timestamped events, mainly exceptions. Here are the root span's raw attributes:

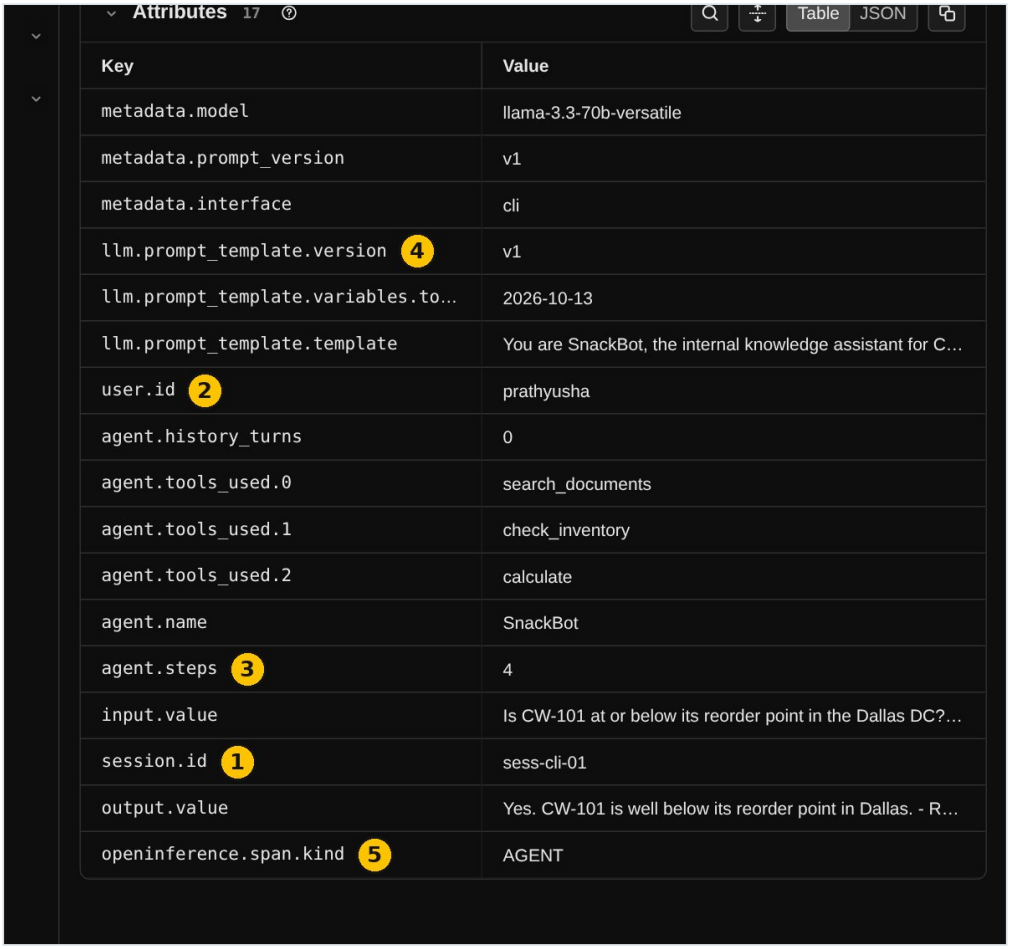


Figure 5. Attributes tab of the AGENT span: the raw key/value pairs.

1	`session.id` from using_attributes in agent.py. It's how Phoenix builds sessions.
2	`user.id`: your Windows username here, because the trace came from cli.py.
3	`agent.steps` = 4 (four LLM calls) and `agent.tools_used.0/1/2`: our own custom attributes. Lists are flattened with .0, .1, .2.
4	`llm.prompt_template.version`: the PROMPT_VERSION. Crucial for comparing prompt changes.
5	`openinference.span.kind` = AGENT: the attribute that decides the badge and the view.

Use Attributes when the Info view doesn't show something, or to check exactly what your code recorded. Use the search box (magnifier) to find a key, and the JSON toggle to copy everything.

5. The LLM span: what the model saw and did

Click the first Completions span. This is the most information-dense screen in Phoenix.

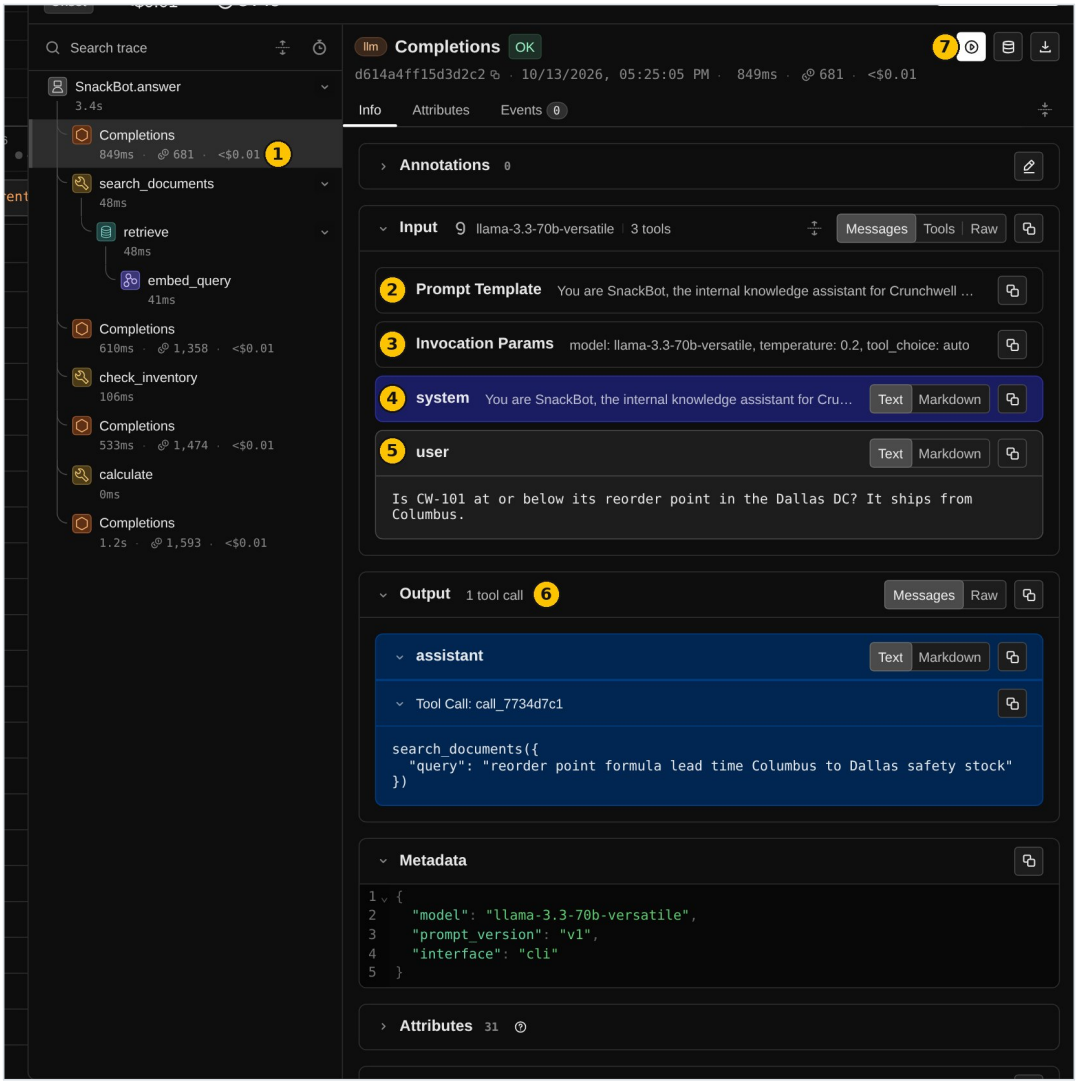


Figure 6. The first LLM span. The model decides to call search_documents.

1	**681 tokens** for this call (656 prompt + 25 completion, see Attributes). Prompt tokens include the system prompt and the tool descriptions, which is why even a short question costs hundreds of tokens.
2	**Prompt Template**: the template with {today} still in it, plus its variables and version. Recorded because agent.py passed prompt_template to using_attributes.
3	**Invocation Params**: model, temperature 0.2, tool_choice auto: the settings this call used. When an answer changes between runs, check these first.
4	**system** message: the full system prompt as actually sent, with the date filled in.

5	**user** message: the question. On later calls you'll also see assistant and tool messages here (see figure 10).
6	**Output: 1 tool call.** The model didn't answer; it asked us to run `search_documents` with the query shown. That query is the model's own rewording of the question. A bad search query here is a common root cause of poor retrieval.
7	**Open in Playground.** Replays this exact call in Phoenix's Playground, where you can edit the prompt or switch models and re-run it. Very useful later for prompt work (Phase 4). The icon next to it adds the span to a dataset (Phase 3).

5.1 The LLM span's raw attributes

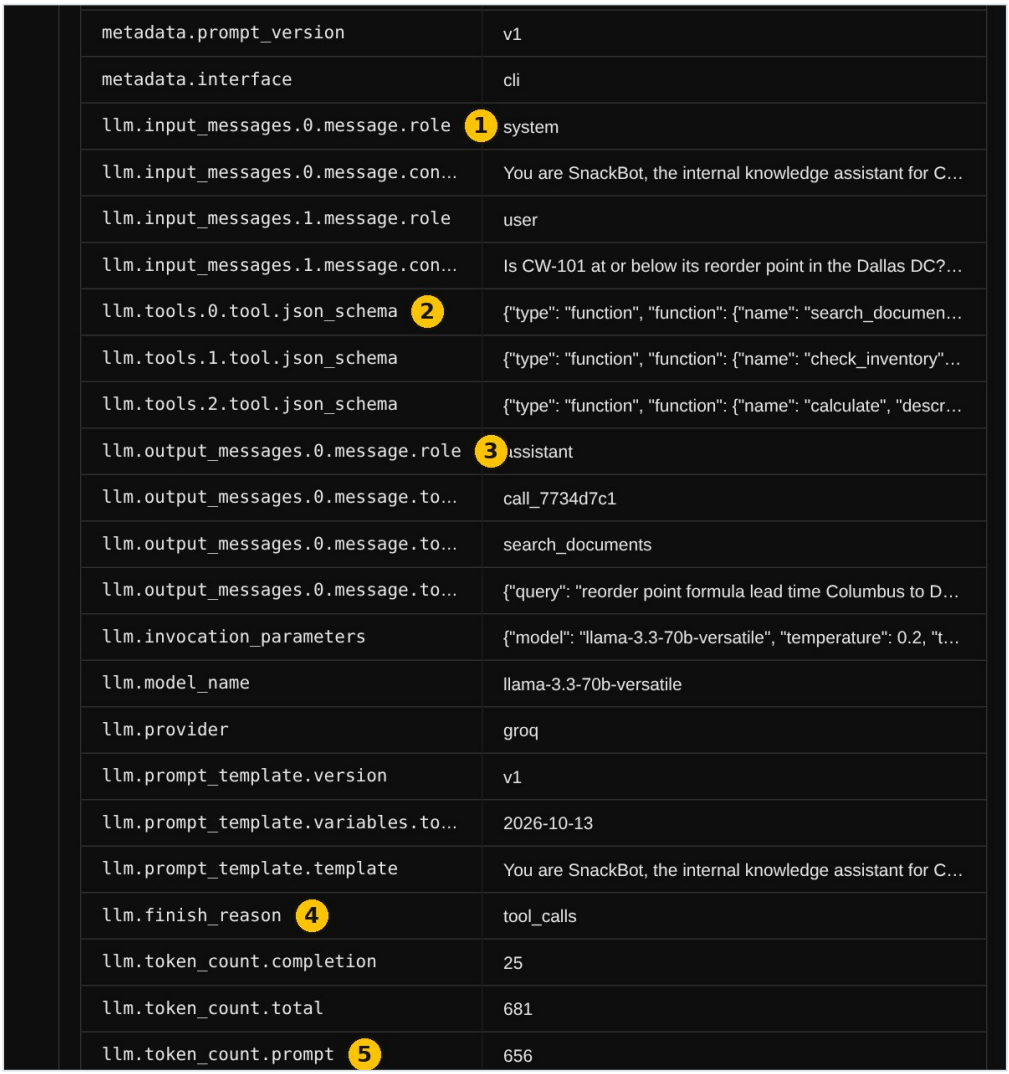


Figure 7. Attributes of the same LLM span, written by the Groq instrumentor.

1	`llm.input_messages.N.message.role / .content`: every message sent, numbered in order.
2	`llm.tools.N.tool.json_schema`: the three tool descriptions sent with every call.

3	`llm.output_messages.0...`: the reply, including tool call name and arguments.
4	`llm.finish_reason` = tool_calls. Learn the values: **stop** (finished normally), **tool_calls** (wants a tool), **length** (cut off by the token limit: the answer is incomplete!).
5	`llm.token_count.prompt / completion / total`: the numbers cost and rate limits are based on.

What to notice

You didn't write any of this code. The Groq instrumentor recorded it all from inside the Groq library. That's the value of auto-instrumentation: complete, standard LLM data for free.

Also note llm.provider = groq and llm.model_name: these drive the cost estimate.

6. Retriever and tool spans

6.1 RETRIEVER: what the search found

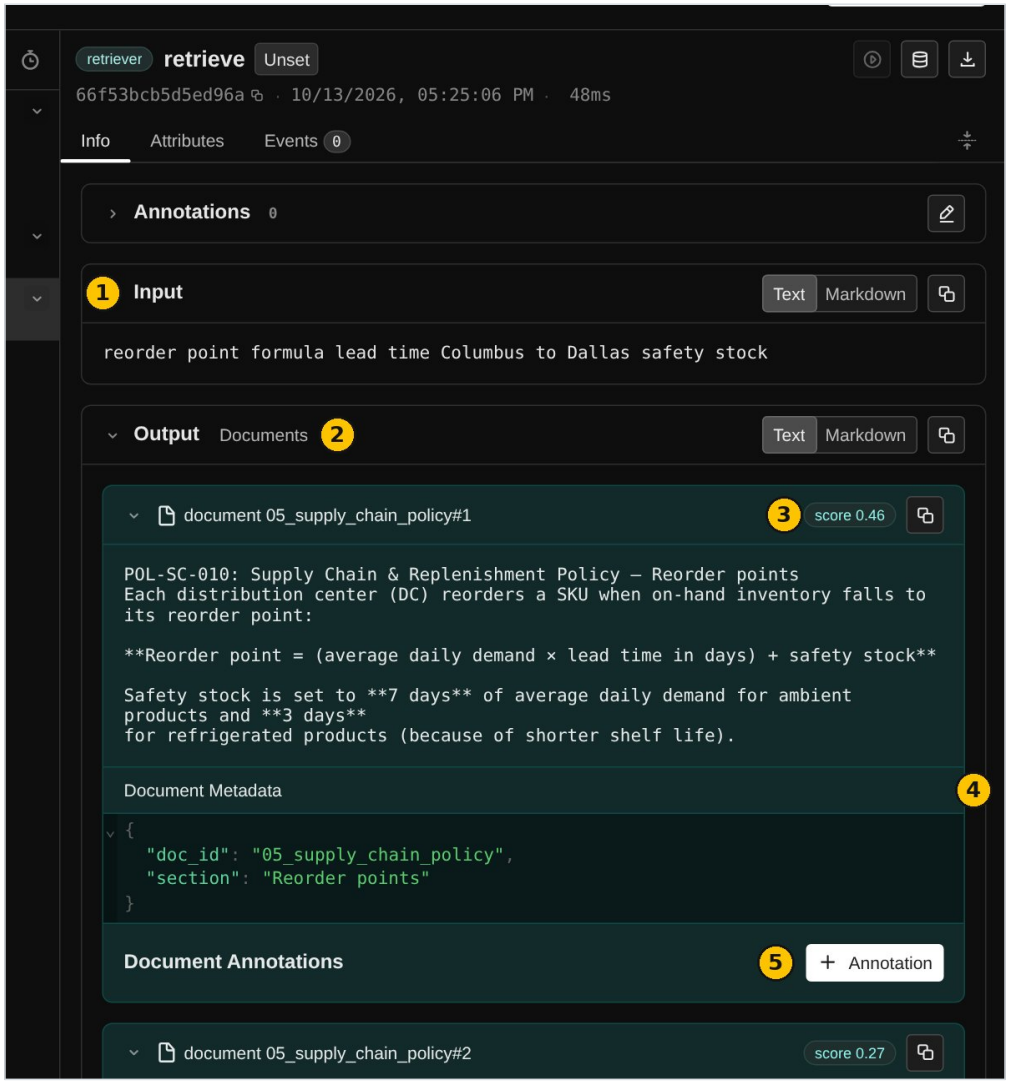


Figure 8. The retrieve span: the ranked documents the search returned.

1	**Input**: the query actually searched, which is the LLM's rewording, not the user's words.
2	**Output: Documents**. Phoenix rebuilt the numbered retrieval.documents.N attributes into a ranked list.
3	**Score**: cosine similarity, highest first. Compare the scores, not just the order: a big gap between #1 and #2 means a confident match; four similar low scores mean nothing matched well.
4	**Document Metadata**: the doc_id and section we stored, so you know where each chunk came from.
5	**+ Annotation**: mark a document as relevant or not. This becomes important in Phase 2, when we measure retrieval quality.

6.2 TOOL: what a tool received and returned

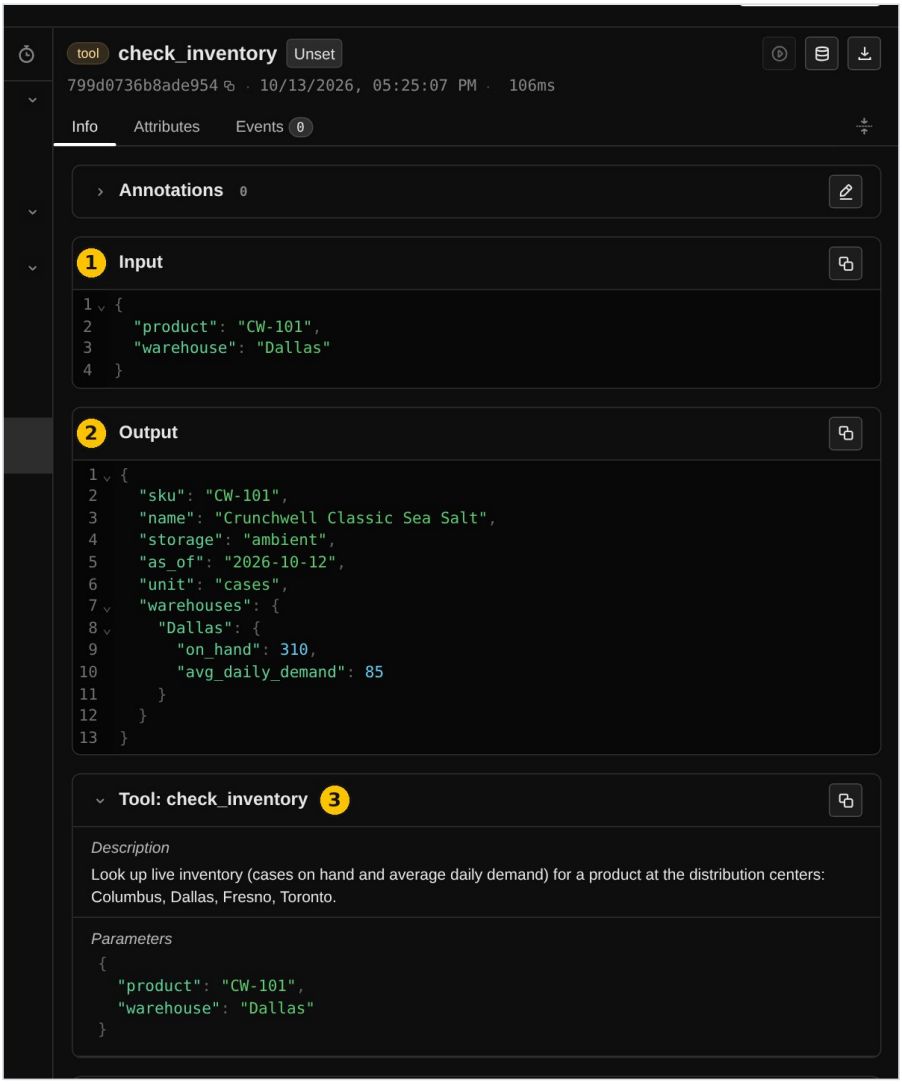


Figure 9. The check_inventory tool span.

1	**Input**: the exact arguments the LLM chose, as JSON. Wrong SKU? Wrong warehouse? You see it here.
2	**Output**: exactly what the tool sent back to the LLM. If the final answer quotes a wrong number, compare it with this.
3	**Tool: check_inventory**: the description and parameters the LLM was given. If the LLM keeps misusing a tool, the description is often the cause.

6.3 The last LLM call: everything the model saw

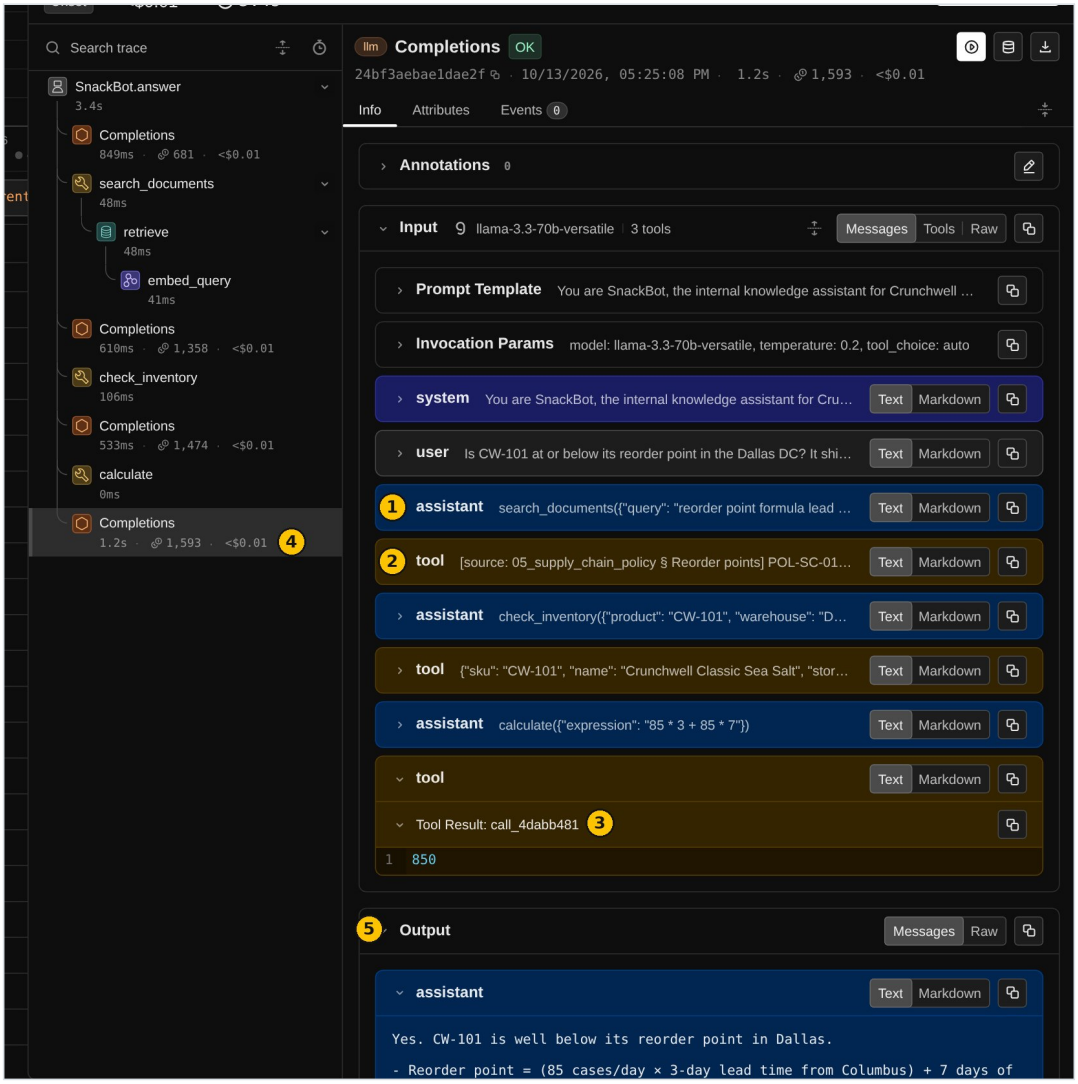


Figure 10. The fourth and final LLM call of the same trace.

1	An **assistant** message: the model's earlier tool request (search_documents), replayed.
2	A **tool** message: what the search returned, sent back to the model. Then the same pattern for check_inventory and calculate.
3	**Tool Result**: the calculator's "850". The model's final answer must be consistent with these results.
4	**1,593 tokens**, up from 681 in the first call. The whole growing conversation is re-sent every step. That's why multi-step questions cost more.
5	**Output**: the final answer (text, so finish_reason = stop).

The single most useful view in Phoenix

The final LLM span's input is *everything the model knew* when it wrote the answer. When an answer is wrong, open this and ask: **was the right information in front of the model?**

If **no**: the problem is upstream (retrieval, tool, or the model's search query). If **yes**: the model ignored or misread it, so look at the prompt, the model choice, or conflicting information in the context. This one question splits almost every RAG bug into two halves.

7. The span kinds at a glance

Badge	What it is	Look at first
agent	The whole question (root span)	Input, output, status, total latency, agent.steps
llm	One call to Groq	Messages in, output (text or tool call), tokens, finish_reason
tool	One tool execution	Input arguments, output, red status
retriever	One document search	Query, document list, scores, sources
embedding	Turning the query into a vector	Duration (slow first call = model loading)
chain, reranker, guardrail, evaluator	Other step types	Appear in later phases

8. Case study: a confidently wrong answer

Question: "*How many unused vacation days can I carry over to next year?*" SnackBot answered **10 days, no expiry**. The correct answer under the current 2026 policy is **5 days, expiring March 31**. Nothing is red; the trace looks healthy. Here's how to find the cause.

Step 1: what did retrieval return?

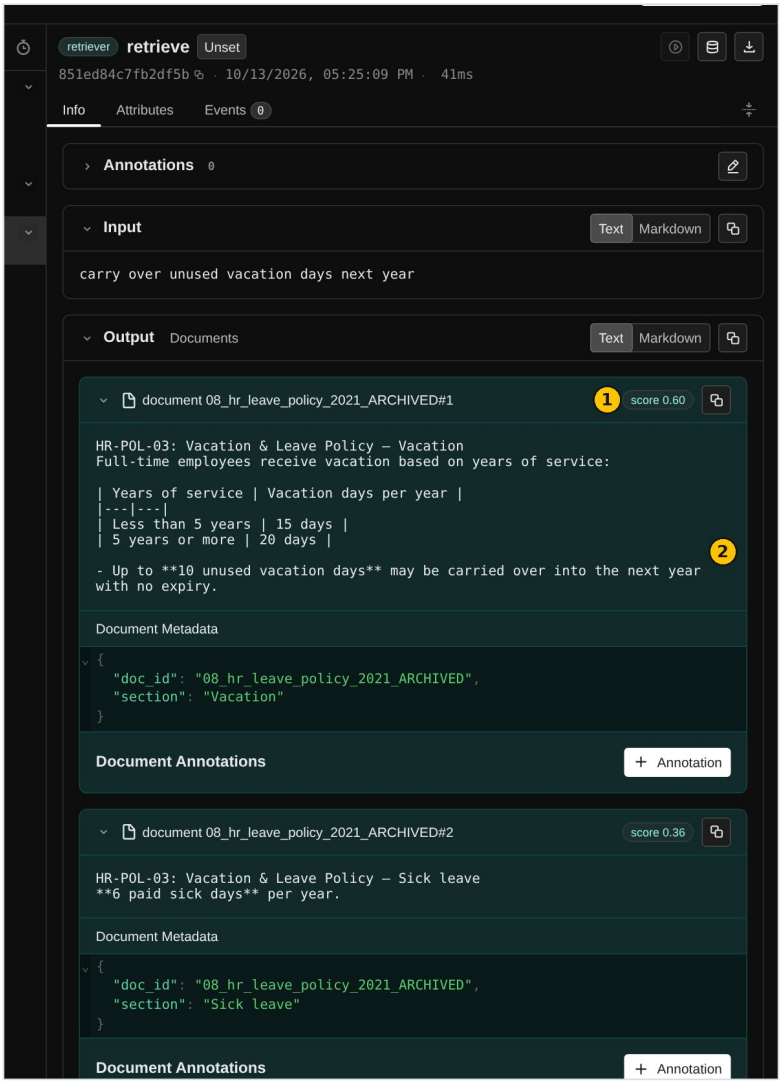


Figure 11. The retrieve span for the carry-over question.

1	**The archived 2021 policy is ranked #1** (score 0.60). Its chunk #2 is also from the archived file. The current 2026 policy only comes third (score about 0.36).
2	The archived text says "*Up to 10 unused vacation days may be carried over... with no expiry*", which is exactly the wrong answer.

Why? The question says "vacation days". The 2021 policy uses the word "vacation"; the 2026 policy calls it "PTO". Similarity search matches words and topics, and has no idea which document is current.

Step 2: what did the model see, and what did it do?

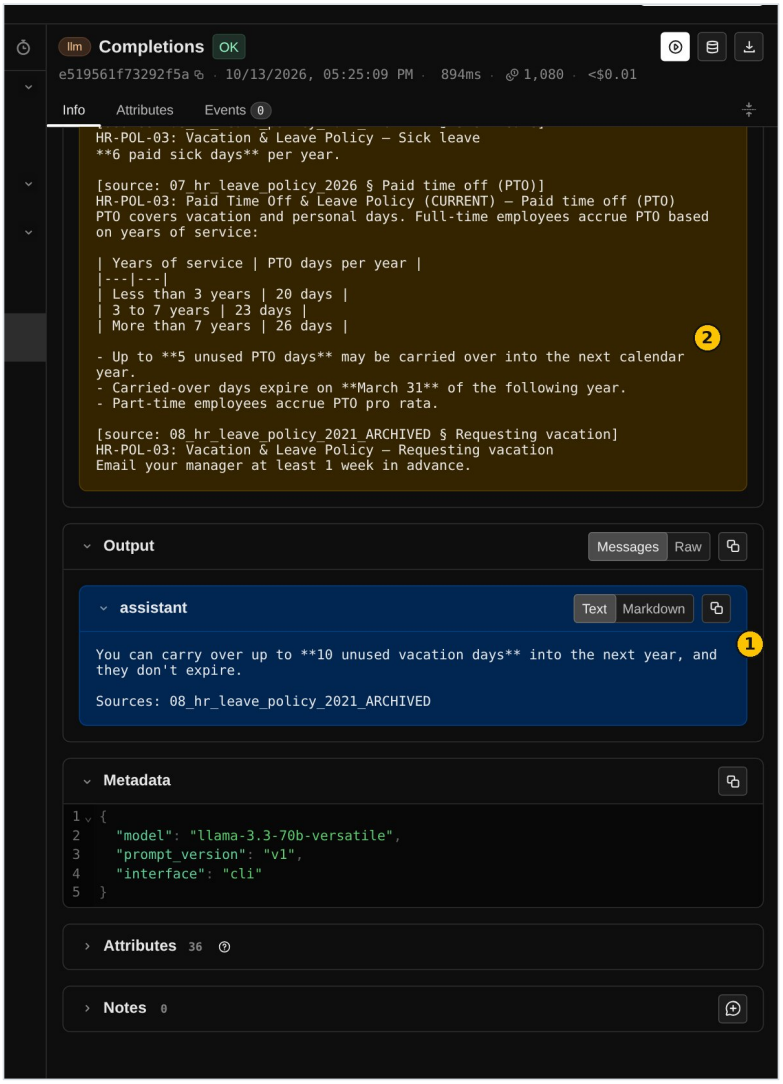


Figure 12. The final LLM span, scrolled to the bottom of its input.

1	The model's answer follows the archived document and even cites it as the source.
2	But the **current** rule ("Up to 5 unused PTO days... expire on March 31") was right there in the context. The system prompt says to prefer the most recent version; the model didn't.

Step 3: the diagnosis

Finding	Layer	Possible fixes (to test in later phases)
Archived doc ranked #1	Retrieval / data	Remove archived docs from the index, or store a status in metadata and filter it out; add synonyms (vacation = PTO)
Model chose the old doc despite the rule	Prompt / model	Clearer prompt rule (e.g. quote both and say which is newer); try a different model

What to notice

Two problems combined to make one wrong answer. Without the trace you'd see only "wrong answer" and might fix only the prompt, leaving the bad retrieval in place.

In Phase 2 you'll catch this automatically with a hallucination/correctness evaluator, and in Phase 3 you'll prove which fix works with an experiment.

9. Case study: errors you'd never see

9.1 A tool error the agent recovered from

Question: "*How much CW-101 stock is in the Houston DC?*" There is no Houston DC.

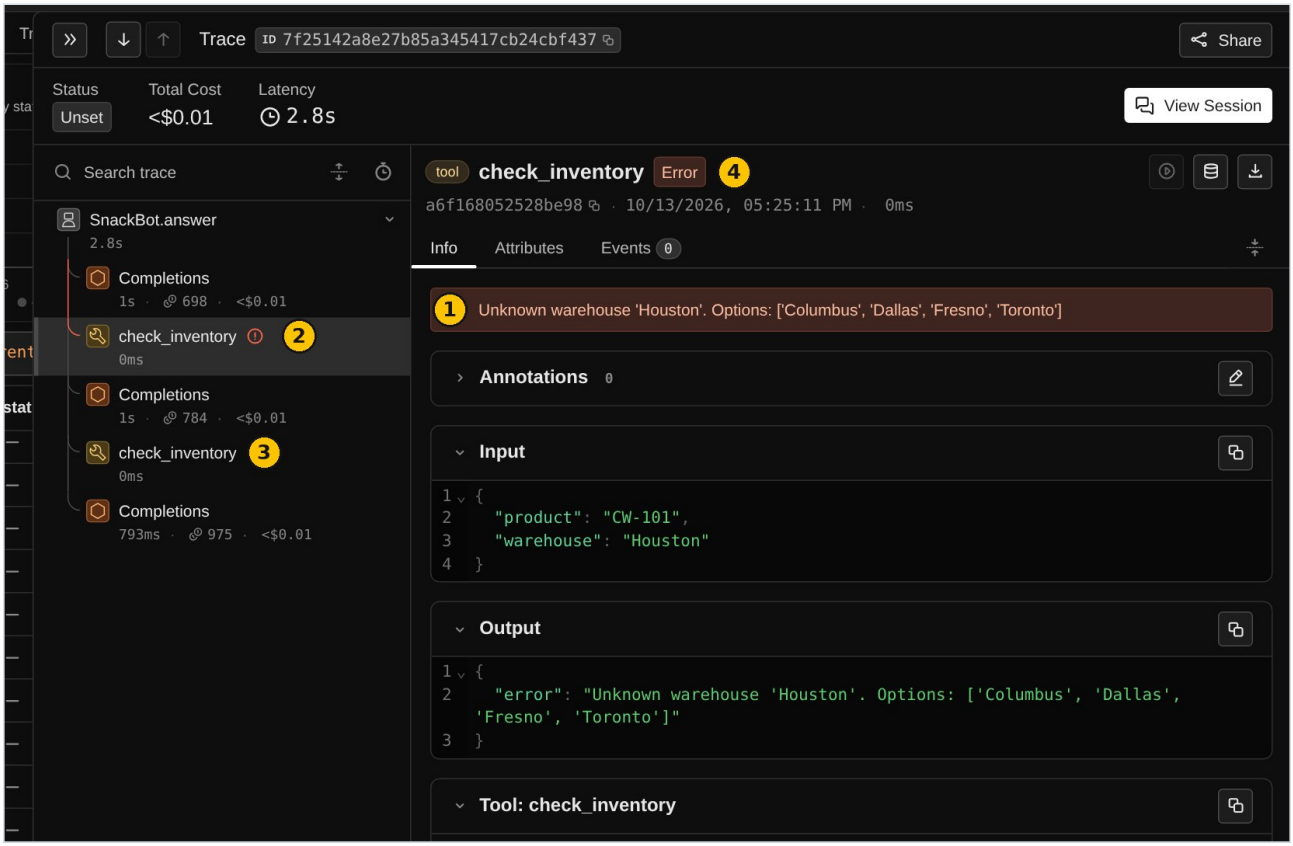


Figure 13. A red TOOL span inside an otherwise successful trace.

1	The error banner: *Unknown warehouse* '*Houston*'. This is the message our code marked with mark_error().
2	The **red icon** in the tree shows which span failed, without opening anything.
3	The model **recovered**: its next step called check_inventory again without a warehouse, then answered helpfully (no Houston DC; Dallas is closest).
4	The span's status badge says **Error**, but the trace status is still Unset, because the question as a whole succeeded.

This is good behaviour, and the trace proves it. Still, if you saw dozens of these, it would tell you users expect a Houston DC, or that the tool description should list the valid warehouses.

9.2 A swallowed LLM error

Here Groq rejected a request with *tool_use_failed*: the model produced a badly formed tool call. The user saw only "Sorry, something went wrong".

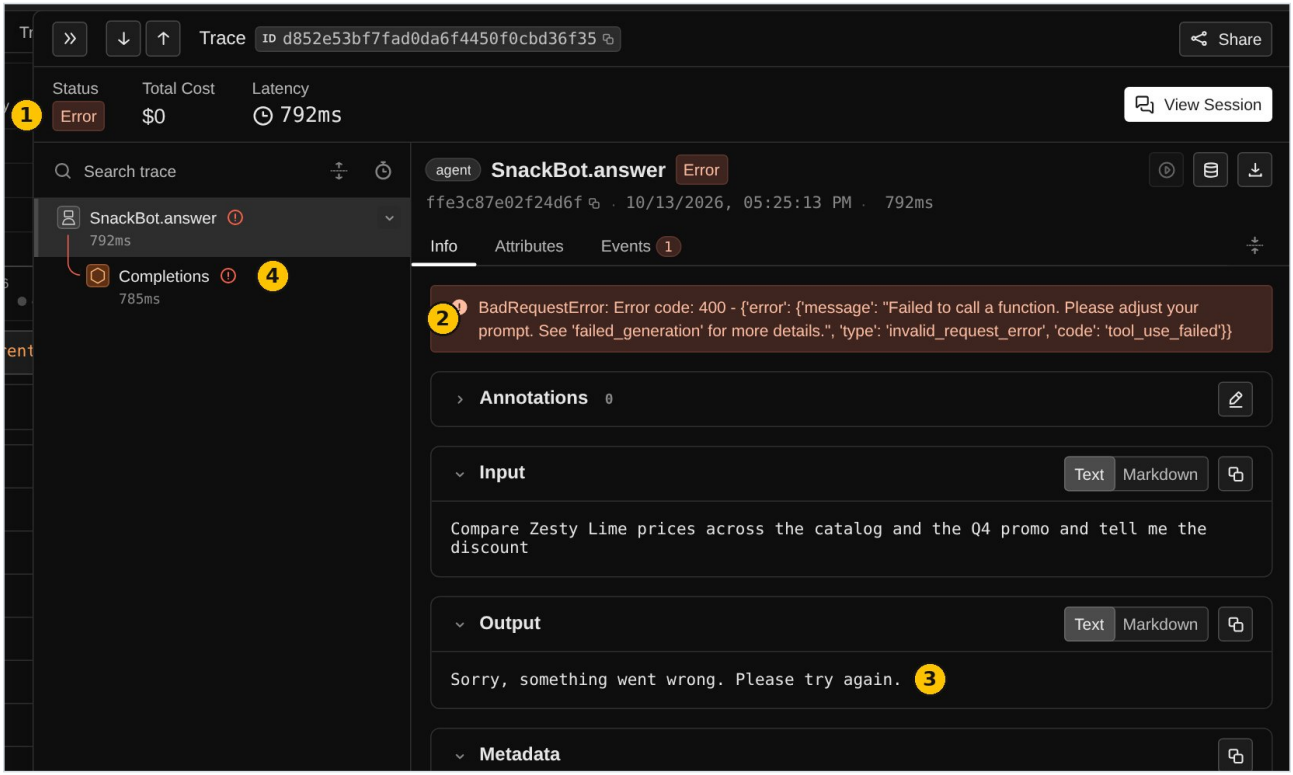


Figure 14. A failed trace: the AGENT span and the LLM span are both red.

1	**Trace status: Error**, and the cost is $0 because the call failed.
2	The real error, recorded by mark_error() in agent.py: BadRequestError 400, code tool_use_failed. The code is the key to the fix (here: usually prompt or model choice for tool calling).
3	What the user got: the polite apology. Without the trace, this is all anyone would know.
4	The LLM span is red too: the Groq instrumentor recorded the failure where it happened.

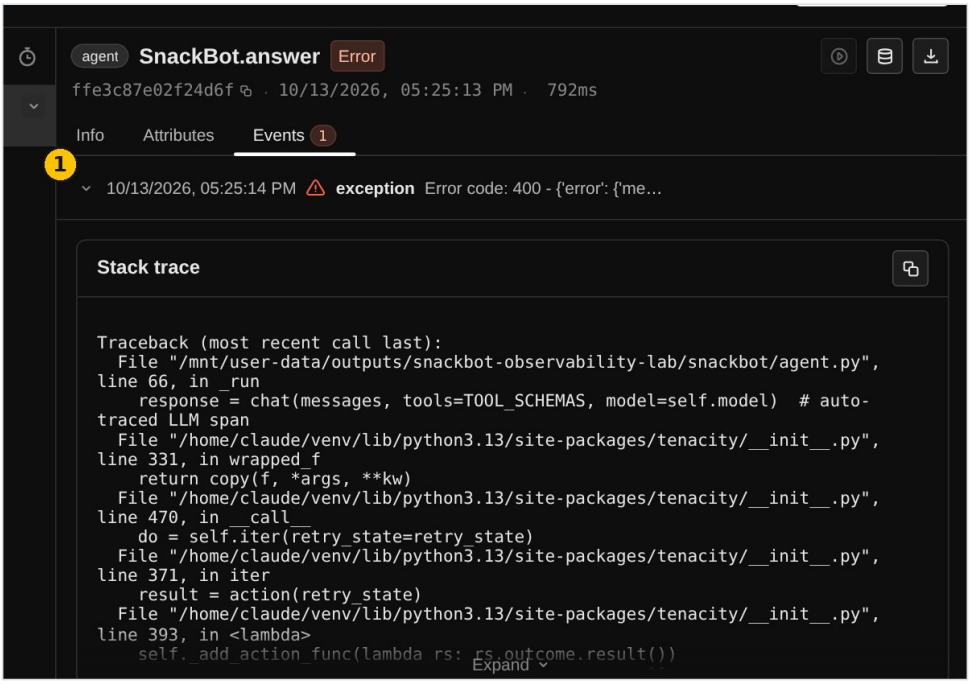


Figure 15. The Events tab of the failed span: the full exception with its stack trace.

1 An **exception event** with time, type and message. Expand it for the **stack trace**: the chain of function calls that led to the error, ending in agent.py's call to chat(). (Your file paths will be Windows paths.)

10. Case study: why was it slow?

Question: "*What is the hotel cap per night in Toronto?*" A simple lookup, but it took 5 seconds.

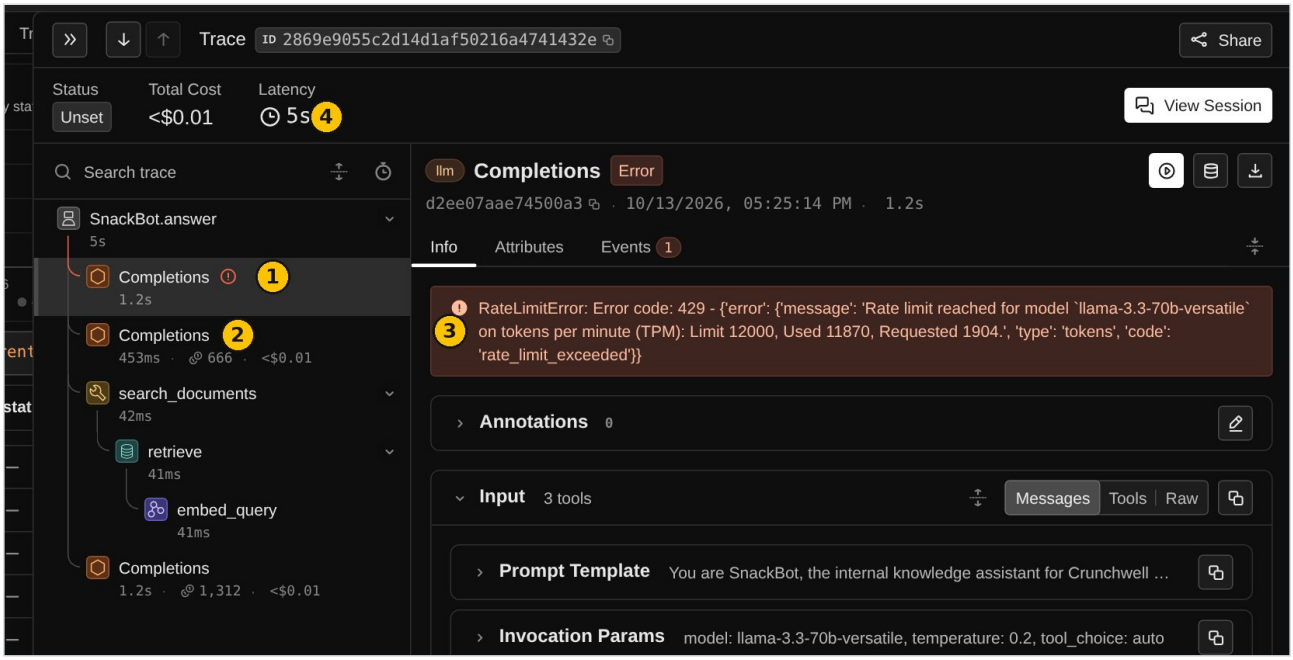


Figure 16. A trace with a rate-limited LLM call followed by a successful retry.

1	The **first LLM call failed** (red). Groq returned 429: rate limit reached on tokens per minute.
2	The **retry** by tenacity in llm.py succeeded. The retry itself is visible as a second LLM span.
3	The exact limit from Groq: 12,000 tokens per minute, 11,870 already used. On the free tier this is the most common cause of slow answers.
4	Total latency 5 s, even though each successful step was fast. The difference is the retry wait (2 s) plus the failed call.

What to notice

To find where time goes, compare the root span's duration with its children's durations. Gaps between children are time spent in your own code or waiting (here, the retry back-off).

If many traces show 429s, you're sending too many tokens per minute: fewer chunks (TOP_K), shorter prompts, or slower batch scripts.

11. Sessions: conversations

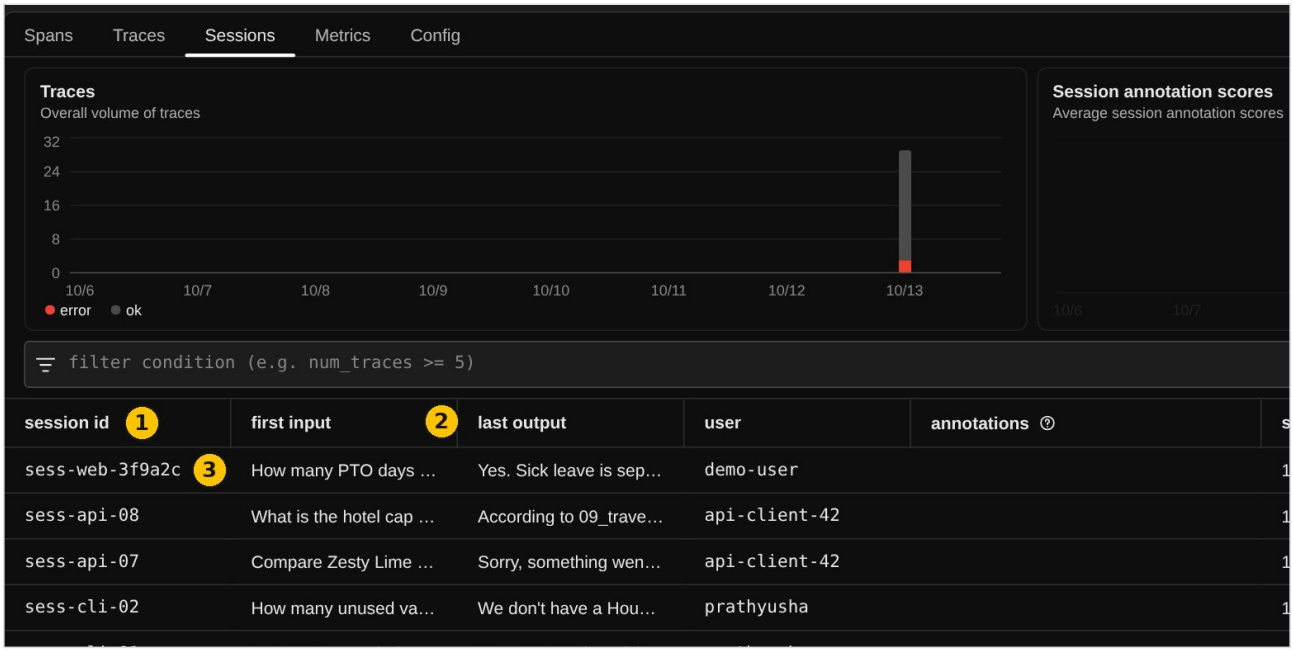


Figure 17. The Sessions tab: one row per conversation.

1	**session id**: the ids our interfaces created (sess-web-… from Streamlit, sess-cli-… from the CLI, sess-api-… from the API).
2	**first input / last output**: how the conversation started and ended, plus user, start and end time, and (further right) token and cost totals.
3	Click a session to open the conversation view.

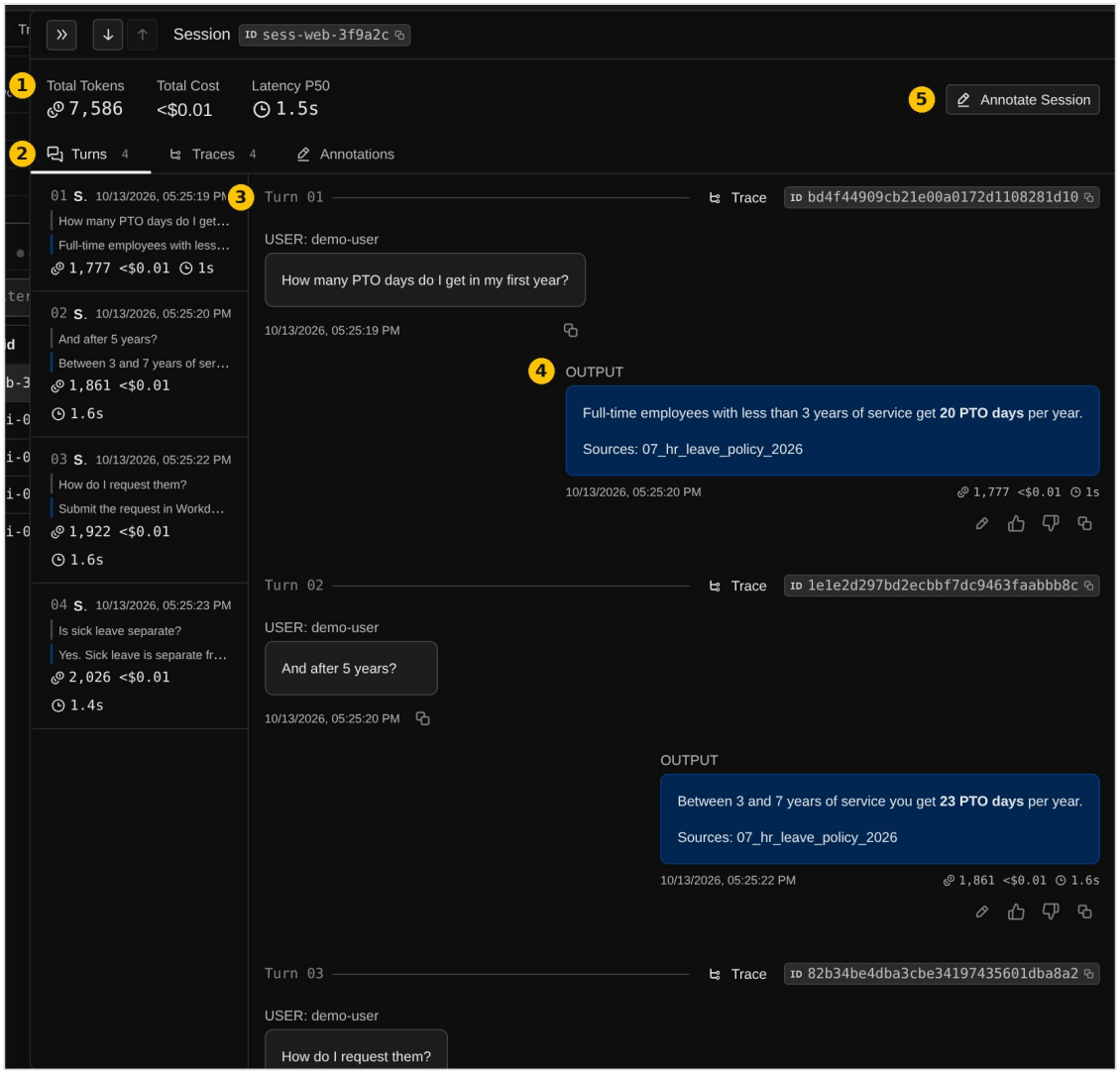


Figure 18. One Streamlit conversation as a chat, with one trace per turn.

1	**Session totals**: 7,586 tokens across 4 turns, cost, and P50 latency.
2	**Turns** list on the left, each with its tokens, cost and latency. Notice tokens per turn: 1,777 → 1,861 → 1,922 → 2,026. They grow because the whole history is re-sent each turn.
3	Each turn links to its **trace**, so you can drill into any message.
4	The conversation as the user saw it. The thumbs up/down and pencil icons let you **annotate** answers: this is human feedback, a big part of Phase 2.
5	**Annotate Session**: give a judgement on the whole conversation.

What to notice

Sessions answer questions single traces can't: did the conversation go off track at turn 3? Did a follow-up ("and after 5 years?") get understood correctly from context? Is token growth going to become a cost problem in long chats?

12. Filters: finding traces that matter

With real traffic you'll have thousands of traces. Filters are how you find the ten that matter. The filter box takes Python-like expressions over span fields and attributes. Press Enter to apply.

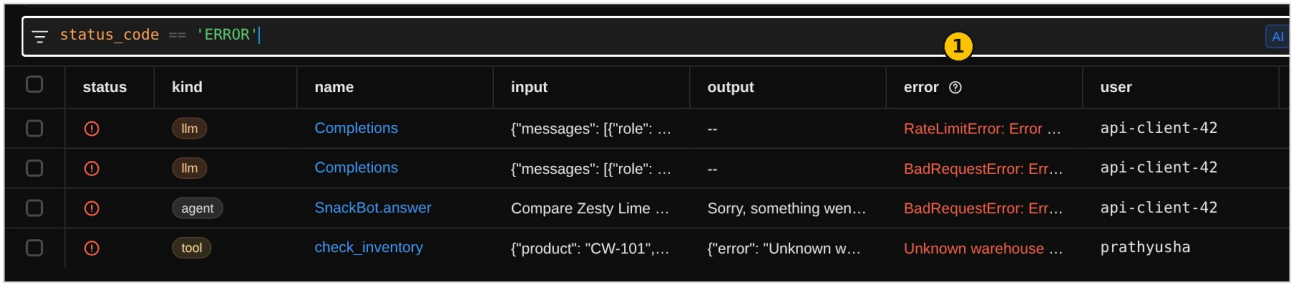


Figure 19. status_code == 'ERROR': every failed span in the project.

Four spans across three traces: the rate-limited and rejected LLM calls, the failed agent run, and the Houston tool error. The **error** column shows each message without opening anything.

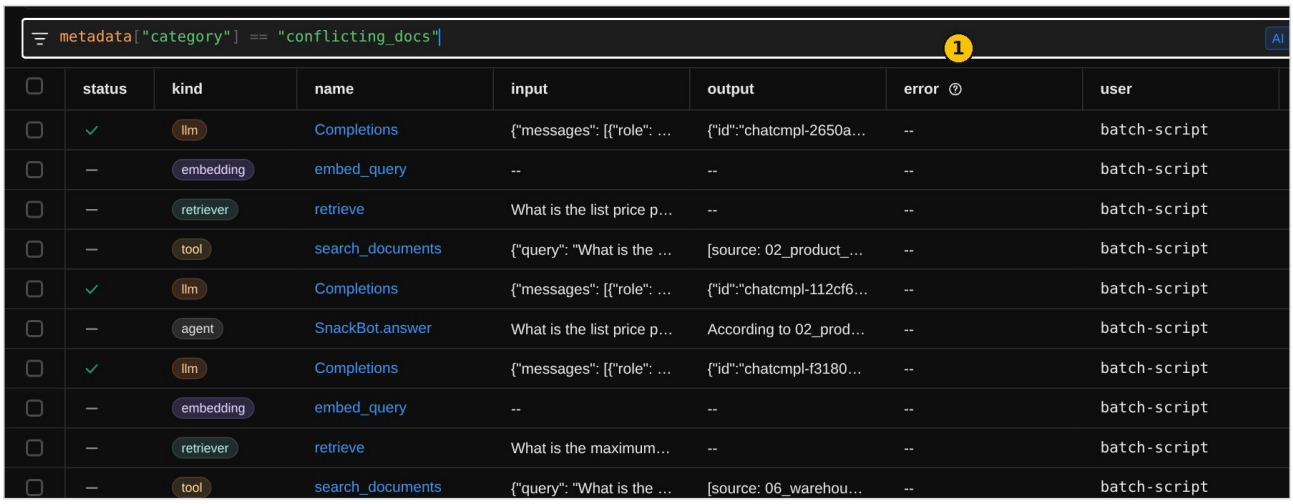


Figure 20. metadata["category"] == "conflicting_docs": every span from one category of test questions.



Figure 21. span_kind == 'LLM' and llm.token_count.total > 1200: only the expensive LLM calls.

12.1 Filters worth memorising

Filter	Finds
parent_id is None	Root spans only: one row per request (the default)
status_code == 'ERROR'	Everything that failed
span_kind == 'TOOL' and name == 'calculate'	Every calculator call: check the expressions
span_kind == 'RETRIEVER'	Every search, to review the queries the model wrote
latency_ms > 5000	Slow spans
llm.token_count.total > 1200	Expensive LLM calls
llm.finish_reason == 'length'	Answers cut off by the token limit
user.id == 'demo-user'	One user's activity
session.id == 'sess-web-3f9a2c'	One conversation's spans
metadata["interface"] == "api"	Traffic from one front end
llm.prompt_template.version == 'v2'	Traces from one prompt version
trace_id == '…'	One trace, e.g. from a bug report
'Houston' in input.value	Spans whose input mentions a word

You can combine conditions with **and** / **or**. The sparkle icon in the filter box turns a plain-English request into a filter, and the box suggests field names as you type.

13. Metrics: the big picture

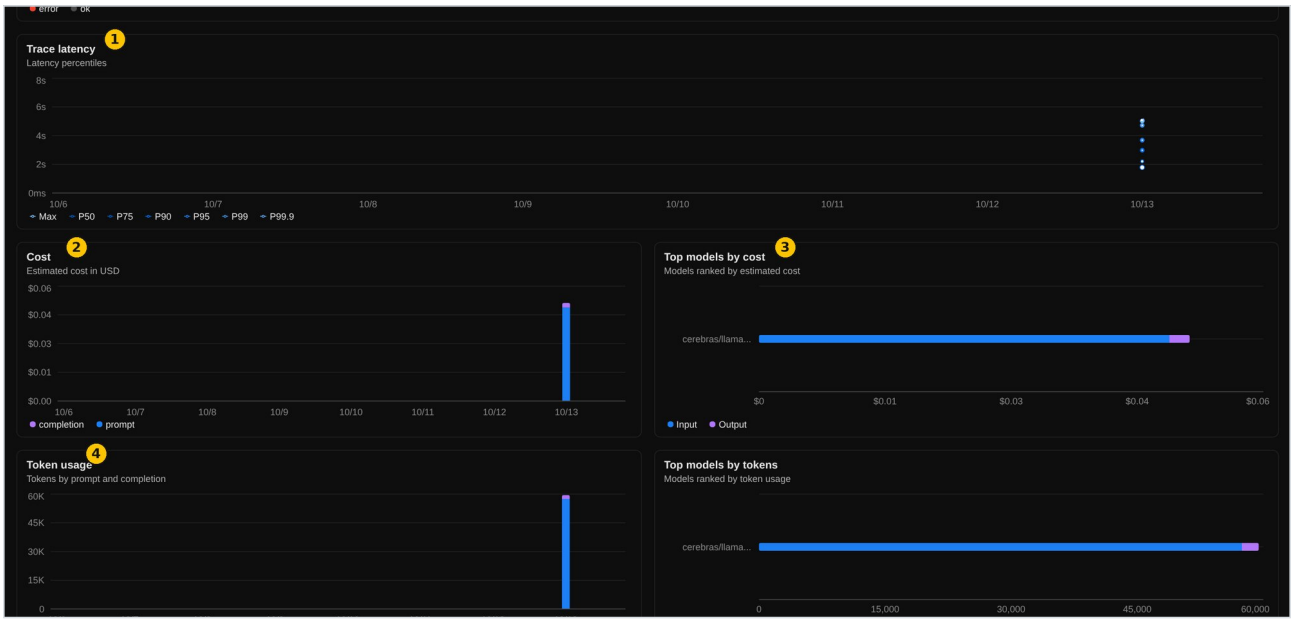


Figure 22. The Metrics tab: charts aggregated over all traces in the time range.

1	**Trace latency percentiles** (P50 to P99.9) over time. Watch the P99 line for tail latency.
2	**Cost** over time, split into prompt and completion.
3	**Top models by cost.** Note it says cerebras/llama...: Phoenix *estimates* cost from a built-in price table by matching the model name, and here it matched another provider's price for the same Llama model. On Groq's free tier you actually pay $0. Treat cost numbers as estimates unless your team has configured prices.
4	**Token usage** over time, prompt vs completion. Prompt tokens dominate in RAG apps because of the retrieved chunks.

In production, these charts are what you'd watch daily: is latency creeping up, did cost jump after a release, did errors spike? A trace explains one request; metrics tell you whether there's a pattern worth investigating.

14. Auto vs full tracing, side by side

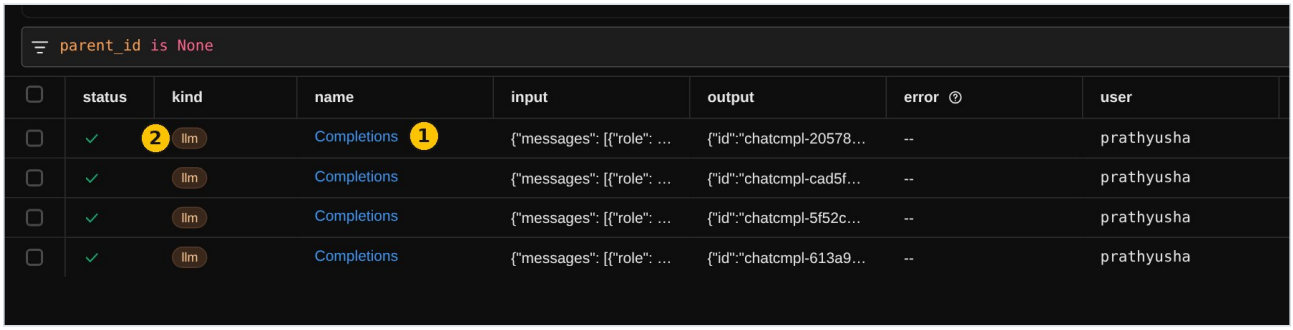


Figure 23. The snackbot-auto project: the same CW-101 question with TRACING=auto.

1	**Four separate rows**, each a Completions LLM span that is its own trace. The same question in the full project was one trace with ten spans.
2	Only **llm** spans exist. No agent, no tools, no retrieval. You can see what the model was sent and said, but not what search returned or what the tools did, and nothing ties the four calls together except timestamps.

This is why real systems combine auto-instrumentation (for LLM calls) with a few manual spans (for your own logic). Most of the value comes from the parent span that groups everything, plus the retriever and tool spans.

15. A checklist for every trace you open

#	Check	Where
1	Is the trace or any span red?	Header status; red icons in the tree
2	Does the tree shape fit the question? (number of LLM calls and tools)	Tree; agent.steps
3	Which step is slowest? Are there gaps between steps?	Durations under each node
4	Did the model search for the right thing?	retrieve span input
5	Were the right documents found, and how confident were the scores?	retrieve Documents
6	Did each tool get sensible arguments and return correct data?	Tool spans input/output
7	Was the right information in the final LLM call's input?	Last LLM span messages
8	Does the answer match that information, and cite the right source?	Root output vs last LLM input
9	How did finish_reason end? (stop / tool_calls / length)	LLM attributes
10	How many tokens, and how did they grow across steps or turns?	Token counts; session view
11	Which prompt version and model produced this?	Metadata; prompt template
12	Is this a one-off or a pattern?	Filters; Metrics tab

16. Other Phoenix features you'll meet soon

Feature	Where	When we use it
Annotations and notes	On any span, trace, session or document	Phase 2: human feedback and labels
Evaluators	Sidebar	Phase 2: LLM-as-judge and code evaluations
Datasets & Experiments	Sidebar; "add to dataset" icon on spans	Phase 3: test sets and A/B comparisons
Playground	Sidebar; button on LLM spans	Phase 4: replay and edit prompts
Prompts	Sidebar	Phase 4: versioned prompt management
Dashboards	Sidebar	Phase 4: production monitoring
Ask PXI	Top right	Any time: ask questions about your traces in plain English
Download span / Share	Trace view buttons	Export a span's data; send a trace link to a colleague

17. Practice exercises

Do these on your own Phoenix with real Groq answers. Each should take 5–15 minutes. Write your findings down; the root-cause notes become your eval ideas in Phase 2.

1. **Read a trace aloud.** Ask the CW-101 question. Without looking at the code, explain every span in the tree to yourself: what ran, why, what went in, what came out.
2. **Token growth.** In that trace, write down the prompt tokens of each LLM call. How much does each step add? Which message added the most?
3. **The carry-over test.** Ask the vacation carry-over question. Using section 8's method, decide: retrieval problem, model problem, both, or neither? (With the real embedding model your ranking may differ.)
4. **Break it on purpose.** Set GROQ_MODEL to a model that doesn't exist, ask a question, and find the exception and its message in the Events tab. Restore the model.
5. **Find a tool error.** Ask about a warehouse that doesn't exist, or a product like "unicorn chips". Find the red span. Did the model recover?
6. **Session view.** Have a 5-turn conversation in Streamlit with follow-ups. Open the session; check the tokens per turn, and whether every follow-up was understood.

7. **Batch and filter.** Run the 20 sample questions. Use filters to answer: how many calculate calls were there? Which question used the most tokens? Did any finish_reason equal 'length'?
8. **Root-cause table.** For every wrong answer in the batch, fill in Lab 7's table from the Phase 1 notes: question, what's wrong, layer, evidence (span + attribute).
9. **Explain auto vs full.** Run one question with TRACING=auto and one with full. Explain in two sentences why full tracing is so much more useful.

17.1 You're ready for Phase 2 when you can...

- open any trace and explain its tree in under a minute;
- find the retrieved documents and judge whether they were relevant;
- tell "the information was missing" apart from "the model ignored it";
- find every failed span in the project with one filter;
- explain why tokens grow inside a trace and across a session;
- say why a trace was slow, using span durations.

> **What Phase 2 adds**
>
> So far *you* judge whether an answer is right by reading traces. Phase 2 makes Phoenix do the judging: evaluators score answers for correctness, relevance and hallucination, and attach those scores to these same traces as annotations. You'll then filter for "all hallucinated answers" the same way you filtered for errors here.